AI AGENT EVALUATION AND OBSERVABILITY PLATFORM FOR RELIABLE RAG-BASED APPLICATIONS

A MINI PROJECT REPORT

Submitted by

SAKTHI MANOJ M (922524118046)
UVANESWARAN B (922524118059)

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER AND COMMUNICATION ENGINEERING

V.S.B. ENGINEERING COLLEGE, KARUR – 639111
(AN AUTONOMOUS INSTITUTION)
ANNA UNIVERSITY :: CHENNAI 600 025

NOVEMBER 2026

BONAFIDE CERTIFICATE

Certified that this project report titled "**AI AGENT EVALUATION AND OBSERVABILITY PLATFORM FOR RELIABLE RAG-BASED APPLICATIONS**" is the bonafide work of **SAKTHI MANOJ M (922524118046)** and **UVANESWARAN B (922524118059)**, carried out under the supervision of the guide named below.

SIGNATURE

Mr. R. R. JEGAN, M.E., (Ph.D.,)

HEAD OF THE DEPARTMENT

Department of CCE

V.S.B. ENGINEERING COLLEGE (AUTONOMOUS),

Karur-639111.

SIGNATURE

Mrs. S.YUVARANI, M.E

PROJECT MENTOR

Department of CCE

V.S.B. ENGINEERING COLLEGE (AUTONOMOUS),

Karur-639111.

Submitted for Anna University Mini-Project Viva-Voce held on

_____

INTERNAL EXAMINER

ACKNOWLEDGEMENT

First and foremost, we express our thanks to our parents for providing us with a very nice environment for doing this project. We wish to express our sincere thanks to our Founder and Chairman **Shri. V. S. BALSAMY, B.Sc., L.L.B.,** for his endeavour in educating us in this premier institution.

We extend our gratitude to **Dr. C. VENNILA, M.E., Ph.D.,** Principal, and **Dr. T. S. KIRUBA SHANKAR M.E., Ph.D.,** Vice Principal, V.S.B. ENGINEERING COLLEGE (AUTONOMOUS), KARUR, for their encouragement and moral support throughout our course.

We express our indebtedness to **Mr.R. R. JEGAN, M.E., ,** Head of the Department of Computer and Communication Engineering, for his/her valuable support.

We are grateful to our Project Supervisor **Mrs. S.YUVARANI, M.E**, Department of Computer and Communication Engineering, for the guidance and suggestions given throughout the course of the project.

Our sincere thanks to all the faculty members of the Department of Computer and Communication Engineering and to our friends for their help in the successful completion of this project work.

Finally, we bow before God, the Almighty, who has always given us strength to complete this work.

ABSTRACT

Large Language Models (LLMs), Retrieval-Augmented Generation (RAG) and tool-using AI agents are increasingly applied to question answering, information retrieval and task automation. However, producing a fluent answer does not guarantee that the answer is correct, grounded in the retrieved evidence or produced through the right sequence of actions. Retrieval may return irrelevant passages, a tool call may fail silently, the model may state unsupported claims, and latency or token usage may grow after a prompt or model change. Evaluating only the final answer by manual inspection makes such failures difficult to detect, locate and repeat.

This project proposes an AI Agent Evaluation and Observability Platform for reliable RAG-based applications. The aim is to provide an integrated environment for testing, evaluating and monitoring RAG-based AI agents using measurable quality, execution and observability metrics. The proposed system executes an agent over a controlled evaluation dataset and records an execution trace for every run, including the query, retrieved chunks and similarity scores, tool calls, prompts, model responses, latency, token usage and errors. An evaluation engine then measures retrieval quality, answer relevance, faithfulness, tool-selection correctness and task completion by combining deterministic checks, model-based evaluation (LLM-as-a-Judge) and a small human-validated subset used for calibration.

The planned architecture uses a React and TypeScript frontend, a FastAPI backend, a LangGraph agent controller, Sentence Transformers with FAISS or Chroma for retrieval, Ollama or Hugging Face models for inference, SQLite for storage and MLflow or Langfuse for tracing. A dashboard will present traces, evaluation metrics, prompt and model comparisons and failure categories, and the same dataset can be re-run as a regression test after any change.

The expected contribution is an integrated, lightweight workflow that links execution tracing, RAG evaluation, response assessment, tool monitoring, experiment comparison and failure analysis. This report describes the proposed design, methodology and test plan; actual implementation results and measured metrics will be added after the experiments are executed.

Keywords: Large Language Model, Retrieval-Augmented Generation, AI Agent, Agent Evaluation, RAG Evaluation, Observability.

TABLE OF CONTENTS

ABSTRACT ..... **iv**

LIST OF FIGURES ..... **vii**

LIST OF TABLES ..... **viii**

LIST OF ABBREVIATIONS ..... **viii**

1 INTRODUCTION **1**

1.1 INTRODUCTION TO ARTIFICIAL INTELLIGENCE AND LARGE LANGUAGE MODELS 1

1.2 OVERVIEW OF RETRIEVAL-AUGMENTED GENERATION (RAG) 1

1.3 INTRODUCTION TO AI AGENTS AND TOOL USE 1

1.4 NEED FOR EVALUATION AND OBSERVABILITY 2

1.5 PROJECT MOTIVATION 2

1.6 PROBLEM STATEMENT 2

1.7 AIM OF THE PROJECT 3

1.8 OBJECTIVES 3

1.9 SCOPE OF THE PROJECT 3

1.10 ORGANIZATION OF THE REPORT 3

2 LITERATURE SURVEY **4**

2.1 INTRODUCTION 4

2.2 RETRIEVAL-AUGMENTED GENERATION FOR KNOWLEDGE-INTENSIVE NLP TASKS 4

2.3 RAGAS: AUTOMATED EVALUATION OF RETRIEVAL AUGMENTED GENERATION 4

2.4 ARES: AN AUTOMATED EVALUATION FRAMEWORK FOR RAG SYSTEMS 4

2.5 RAGCHECKER: FINE-GRAINED DIAGNOSIS OF RAG 5

2.6 AGENTBENCH: EVALUATING LLMS AS AGENTS 5

2.7 LLM-AS-A-JUDGE 5

2.8 SELF-RAG: RETRIEVE, GENERATE AND CRITIQUE THROUGH SELF-REFLECTION 5

2.9 MAESTRO: TESTING, RELIABILITY AND OBSERVABILITY OF AGENT SYSTEMS 6

2.10 COMPARATIVE SUMMARY 6

2.11 LITERATURE SYNTHESIS AND RESEARCH GAP 6

3 SYSTEM ANALYSIS **7**

3.1 OVERVIEW OF THE EXISTING SYSTEM 7

3.2 EXISTING-SYSTEM WORKFLOW 7

3.3 DISADVANTAGES OF THE EXISTING SYSTEM 7

3.4 PROPOSED SYSTEM 7

3.5 ADVANTAGES OF THE PROPOSED SYSTEM 7

3.6 FUNCTIONAL REQUIREMENTS 8

3.7 NON-FUNCTIONAL REQUIREMENTS 8

3.8 USER ROLES AND SYSTEM USE 8

4 SYSTEM SPECIFICATION **9**

4.1 HARDWARE REQUIREMENTS 9

4.2 SOFTWARE REQUIREMENTS 9

4.3 FRONTEND REQUIREMENTS 9

4.4 BACKEND REQUIREMENTS 10

4.5 AI AND RAG COMPONENTS 10

4.6 DATABASE AND LOGGING REQUIREMENTS 10

4.7 DEVELOPMENT AND OPTIONAL DEPLOYMENT TOOLS 10

5 IMPLEMENTATION AND METHODOLOGY **11**

5.1 OVERALL METHODOLOGY 11

5.2 DATASET AND DOCUMENT PREPARATION 11

5.3 TEXT EXTRACTION, CLEANING AND CHUNKING 12
5.4 EMBEDDING GENERATION AND INDEXING 12
5.5 RETRIEVAL PROCESS 12
5.6 AI AGENT EXECUTION 13
5.7 TOOL EXECUTION 13
5.8 TRACE COLLECTION 13
5.9 EVALUATION ENGINE 14
5.10 DASHBOARD AND EXPERIMENT COMPARISON 14
6 SYSTEM DESIGN 15
6.1 INTRODUCTION TO SYSTEM DESIGN 15
6.2 OVERALL SYSTEM ARCHITECTURE 15
6.3 SYSTEM ARCHITECTURE / BLOCK DIAGRAM 15
6.4 CONTEXT-LEVEL DATA FLOW DIAGRAM (DFD LEVEL 0) 16
6.5 DETAILED DATA FLOW DIAGRAM (DFD LEVEL 1) 17
6.6 USE CASE DIAGRAM 18
6.7 SEQUENCE DIAGRAM 19
6.8 ACTIVITY DIAGRAM 20
6.9 ENTITY RELATIONSHIP DIAGRAM (ERD) 21
6.10 EVALUATION PIPELINE AND TRACE LIFECYCLE 22
6.11 DATABASE DESIGN 23
6.12 MODULE DESCRIPTION 24
7 SYSTEM TESTING AND EVALUATION 25
7.1 TESTING OVERVIEW 25
7.2 UNIT TESTING 25
7.3 INTEGRATION TESTING 25
7.4 FUNCTIONAL TESTING 25
7.5 SYSTEM TESTING 26
7.6 EVALUATION METRICS 26
7.7 TEST-CASE DESIGN 26
7.8 FAILURE ANALYSIS 27
7.9 RESULT PRESENTATION 27
8 FEASIBILITY STUDY, WORK PLAN AND LIMITATIONS 28
8.1 TECHNICAL FEASIBILITY 28
8.2 ECONOMIC FEASIBILITY 28
8.3 OPERATIONAL FEASIBILITY 28
8.4 DATA AND RESOURCE REQUIREMENTS 28
8.5 TWELVE-WEEK WORK PLAN 28
8.6 SCOPE AND LIMITATIONS 29
8.7 RISK CONSIDERATIONS 29
9 CONCLUSION AND FUTURE ENHANCEMENT 30
9.1 CONCLUSION 30
9.2 FUTURE ENHANCEMENT 30
APPENDIX 31
A. ADDITIONAL EVALUATION METRIC DEFINITIONS 31
B. DATASET FIELD DEFINITIONS 31
C. SAMPLE TRACE STRUCTURE 31
D. IMPLEMENTATION SCREENSHOTS (TO BE INSERTED) 31
REFERENCES 32
BIBLIOGRAPHY 32

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
5.1	Methodology lifecycle of the proposed platform	11
5.2	RAG pipeline: offline indexing and online query-time retrieval	12
5.3	Agent execution flow and trace-collection record	13
5.4	Illustrative dashboard layout (design mock-up, not an implementation screenshot)	14
6.1	Layered architecture of the platform	15
6.2	System architecture / block diagram	16
6.3	Context-level data flow diagram (DFD Level 0)	17
6.4	Detailed data flow diagram (DFD Level 1)	18
6.5	Use case diagram	19
6.6	Sequence diagram of a RAG and agent execution	20
6.7	Activity diagram with normal and failure paths	21
6.8	Entity relationship diagram (proposed logical design)	22
6.9	Evaluation and observability lifecycle	23
8.1	Planned twelve-week project work plan	29

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	Comparison of the reviewed studies	6
3.1	Functional requirements	8
4.1	Hardware requirements	9
4.2	Core software stack	9
4.3	Alternatives and optional tools	10
5.1	Planned categories of evaluation queries	11
6.1	Proposed database tables	23
6.2	Modules of the platform	24
7.1	Proposed unit test cases	25
7.2	Proposed integration test cases	25
7.3	Proposed functional test cases	25
7.4	Evaluation metrics, purpose and limitations	26
7.5	Proposed evaluation test cases (structure)	26
7.6	Failure categories	27
8.1	Planned twelve-week work schedule	28
8.2	Risks and mitigation	29
0.1	Fields of an evaluation test case	31

LIST OF ABBREVIATIONS

S.NO.	ABBREVIATION	EXPANSION
1.	AI	Artificial Intelligence
2.	API	Application Programming Interface
3.	CI/CD	Continuous Integration / Continuous Delivery
4.	DFD	Data Flow Diagram
5.	ERD	Entity Relationship Diagram
6.	FAISS	Facebook AI Similarity Search
7.	FK	Foreign Key
8.	JSON	JavaScript Object Notation
9.	LLM	Large Language Model
10.	PK	Primary Key
11.	RAG	Retrieval-Augmented Generation
12.	REST	Representational State Transfer
13.	SQL	Structured Query Language
14.	UML	Unified Modeling Language

CHAPTER 1
INTRODUCTION

1.1 INTRODUCTION TO ARTIFICIAL INTELLIGENCE AND LARGE LANGUAGE MODELS

Artificial Intelligence (AI) is the branch of computer science concerned with building systems that perform tasks normally associated with human intelligence, such as understanding language, reasoning over information and making decisions. Recent progress in deep learning has produced Large Language Models (LLMs), neural networks trained on very large text collections that can answer questions, summarise documents, generate program code and follow natural-language instructions.

Because an LLM stores its knowledge implicitly in its parameters, its answers reflect only the data available when it was trained. The model can also produce fluent statements that are not factually supported, a behaviour commonly described as hallucination. These characteristics make LLMs powerful but difficult to trust without additional checking.

1.2 OVERVIEW OF RETRIEVAL-AUGMENTED GENERATION (RAG)

Retrieval-Augmented Generation (RAG) combines a generative language model with an external, searchable knowledge source. The original RAG work combined the parametric memory of a language model with a non-parametric memory, a retrievable document index, to improve knowledge-intensive generation [1]. In a typical application, documents are divided into chunks, converted into vector embeddings and stored in a vector index. At query time the question is embedded, the most similar chunks are retrieved and supplied to the model as context from which the answer is generated.

RAG allows an application to use private or recent documents without retraining the model and to return the supporting sources together with the answer. However, the quality of the final answer now depends on two separate stages, retrieval and generation, and either of them can fail independently of the other.

1.3 INTRODUCTION TO AI AGENTS AND TOOL USE

An AI agent extends a language model with the ability to decide on a sequence of actions. Instead of producing a single response, an agent may analyse the request, decide whether retrieval is required, call external tools such as a search service, a calendar or a college database, observe the result and then compose the final answer. Orchestration frameworks such as LangGraph represent this behaviour as a graph of steps that share state.

Agent behaviour is more difficult to assess than a single model response, because correctness depends on the tool that was selected, the arguments passed to it, the order of the steps and the handling of failures. AgentBench demonstrated the need for quantitative evaluation of LLMs acting as agents in interactive environments [5].

1.4 NEED FOR EVALUATION AND OBSERVABILITY

Evaluating only the final answer is insufficient for a complex AI agent. A correct-looking answer may have been produced from irrelevant documents, an incorrect answer may have been caused by a failed tool call rather than by the model, and a slow response may be due to retrieval, generation or tool latency. The following problems cannot be diagnosed from the final text alone:

- Inaccurate answers that cannot be traced to the retrieval, tool or generation step responsible.
- Irrelevant retrieved documents that mislead the model or are ignored by it.
- Hallucinations, that is, claims in the answer that are not supported by the retrieved context.
- Failed or incorrect tool calls, including wrong tool selection and wrong arguments.
- High latency in individual steps and growth in token usage and cost.
- Difficulty in comparing prompts or models fairly, because the same questions are not re-run under the same conditions.

Evaluation measures how well the system performs against defined criteria, whereas observability records what happened inside the system during each execution so that the cause of a poor result can be located. Both are required for reliable RAG-based applications.

1.5 PROJECT MOTIVATION

Academic and small-team projects commonly demonstrate a chatbot that answers questions about a collection of documents. Such systems are usually tested manually with a handful of questions. After a prompt, model or chunk-size change there is no repeatable way to know whether the application has improved or regressed. Existing research provides RAG evaluation metrics [2], [3], [4], agent benchmarks [5] and observability studies [8], but a student team has to combine these ideas by hand.

The motivation for this project is to build a lightweight platform that can be run locally and that treats evaluation and tracing as first-class parts of the application rather than as afterthoughts.

1.6 PROBLEM STATEMENT

Current AI-agent and RAG applications are often evaluated using manual inspection or a small number of example queries. Such methods make it difficult to systematically identify incorrectly retrieved information, unsupported or hallucinated statements, incorrect tool selection, poor answer relevance, failures during multi-step execution, high response latency, model and prompt performance differences, and regression after the system is modified. There is therefore a need for an integrated platform that can execute an AI agent, collect its execution trace, evaluate its behaviour and visualise the results in one system.

1.7 AIM OF THE PROJECT

To design and develop an integrated platform for testing, evaluating and monitoring the performance and reliability of RAG-based AI agents using measurable quality, execution and observability metrics.

1.8 OBJECTIVES

- To implement a RAG-based AI-agent application.
- To design a controlled evaluation dataset containing normal, ambiguous, irrelevant and challenging queries.
- To capture complete execution traces of each agent run.
- To evaluate retrieval quality and generated-answer quality.
- To measure agent behaviour such as tool selection and task completion.
- To monitor latency, token usage, failures and execution steps.
- To compare different models and prompt versions.
- To identify failure points in the RAG and agent pipeline.
- To provide evaluation results through a web-based dashboard.
- To support regression testing after changes to prompts, models or retrieval settings.

1.9 SCOPE OF THE PROJECT

The scope covers RAG-based AI agents built over a small or medium document collection, evaluation datasets of up to a few hundred test cases, local or API-based language models, tool-use evaluation, retrieval and response evaluation, trace collection, prompt and model comparison, and dashboard-based analysis. Multi-agent systems, production-scale monitoring, security testing and large public benchmarks are outside the present scope and are listed as future enhancements in Chapter 9.

This report describes the design, methodology and test plan of the proposed platform. Components are described as planned design elements unless a statement explicitly says otherwise, and no measured results are reported.

1.10 ORGANIZATION OF THE REPORT

Chapter 1 introduces the domain, the problem, the aim and the objectives. Chapter 2 reviews the related literature and identifies the research gap. Chapter 3 analyses the existing and proposed systems and lists the requirements. Chapter 4 gives the hardware and software specification. Chapter 5 explains the methodology and the implementation pipeline. Chapter 6 presents the system design with architecture, data flow, UML and database diagrams. Chapter 7 describes the testing and evaluation plan. Chapter 8 discusses feasibility, the work plan and limitations. Chapter 9 concludes the report and lists future enhancements. The appendices and references follow.

CHAPTER 2
LITERATURE SURVEY

2.1 INTRODUCTION

The proposed platform draws on four lines of research: retrieval-augmented generation, automated evaluation of RAG systems, evaluation of LLM agents, and observability of agent systems. This chapter reviews eight studies [1]-[8]. For each study the research problem, method, contribution, limitations and relevance to this project are discussed. Statements about a study are limited to what is reported in its title, abstract and the project source document; details that could not be confirmed are marked for verification.

2.2 RETRIEVAL-AUGMENTED GENERATION FOR KNOWLEDGE-INTENSIVE NLP TASKS

Lewis et al. [1] address the limitation that pre-trained language models store knowledge in their parameters, which makes it hard to update, inspect or attribute that knowledge. They propose RAG models that combine a pre-trained sequence-to-sequence generator with a dense retriever over an external document index, so that retrieved passages condition the generated output. The study established the retrieve-then-generate pattern now used in most document-grounded assistants. Its limitations, from the perspective of this project, are that it focuses on building the generator and retriever rather than on measuring them in deployment, and that generated text can still diverge from the retrieved evidence. It is the foundation for the RAG engine of the proposed system.

2.3 RAGAS: AUTOMATED EVALUATION OF RETRIEVAL AUGMENTED GENERATION

Es et al. [2] address the difficulty of evaluating RAG pipelines, where the retriever, the generator and the use of context all affect quality and where reference answers are often unavailable. RAGAs proposes reference-free metrics that assess aspects such as faithfulness of the answer to the context, relevance of the answer and relevance of the retrieved context, using a language model to judge them. Its main contribution is a practical set of metric definitions that can be computed without human-labelled answers. The metrics depend on the judging model and prompt, so their scores should be validated on a sample. The proposed Evaluation Engine adopts these dimensions for its model-based metrics.

2.4 ARES: AN AUTOMATED EVALUATION FRAMEWORK FOR RAG SYSTEMS

Saad-Falcon et al. [3] also target automated RAG evaluation, with emphasis on context relevance, answer faithfulness and answer relevance. ARES prepares lightweight judge models using synthetic data and uses a small set of human annotations to improve the reliability of the resulting scores. Its contribution is the combination of automated judging with limited human validation. The approach needs additional preparation, such as

synthetic data generation and annotation, and may need adaptation to a new domain. The idea of calibrating automated metrics with a small human-reviewed set is directly reused in this project.

2.5 RAGCHECKER: FINE-GRAINED DIAGNOSIS OF RAG

Ru et al. [4] observe that a single overall score hides whether a poor answer was caused by the retriever or by the generator. RAGChecker proposes a fine-grained evaluation framework with diagnostic metrics for the retrieval and generation modules. Its contribution is the separation of the two failure sources, which supports targeted improvement. Fine-grained analysis increases the computational cost of evaluation and relies on automated claim-level judgements. The proposed platform follows this principle by storing component-level metrics and failure categories instead of one aggregated number.

2.6 AGENTBENCH: EVALUATING LLMS AS AGENTS

Liu et al. [5] study how well LLMs behave as agents that act in interactive environments. AgentBench provides a multi-environment benchmark and analyses the failure modes of the evaluated models, including weaknesses in reasoning, decision making and instruction following. It shows that agent quality has to be measured over multi-step interaction rather than from static text. The benchmark environments are fixed and may not match the tools and documents of a particular application. In this project, the idea of measuring behaviour over steps motivates tool-selection accuracy, task completion rate and step efficiency.

2.7 LLM-AS-A-JUDGE

Zheng et al. [6] examine the use of strong LLMs as judges of other models' responses, using the MT-Bench and Chatbot Arena resources. The study reports that LLM judges can approximate human preferences and discusses known biases and limitations of such judgements. The contribution is both the justification for automated judging at scale and the warning that judge outputs are not ground truth. Consequently, the proposed platform uses LLM-based evaluation only as one component, combined with deterministic checks and a human-validated subset.

2.8 SELF-RAG: RETRIEVE, GENERATE AND CRITIQUE THROUGH SELF-REFLECTION

Asai et al. [7] address indiscriminate retrieval and the lack of verification of generated content. Self-RAG trains a language model to decide when to retrieve and to critique its own retrieved passages and generations through self-reflection. Its contribution is the idea of a system that retrieves, generates and critiques. The method modifies and trains the model itself, which is beyond the scope of a platform that evaluates existing models from outside. It is relevant because it shows that retrieval need and answer support are separate judgements that can be assessed.

2.9 MAESTRO: TESTING, RELIABILITY AND OBSERVABILITY OF AGENT SYSTEMS

According to the project source document, Ma et al. [8] study testing, reliability and observability of LLM-based multi-agent systems and consider execution traces, latency, cost and failures. The work is closely related to the present project because it connects evaluation with observability. Its specific methods and findings should be confirmed against the original paper before final submission [CONFIRM DETAILS FROM THE ORIGINAL PAPER]. The proposed platform targets a smaller, single-agent RAG setting and does not claim to extend that work.

2.10 COMPARATIVE SUMMARY

Author / work	Year	Focus	Main contribution	Relevance to the proposed platform
Lewis et al. [1]	2020	RAG	Retriever plus generator for knowledge-intensive tasks	Basis of the RAG engine
Es et al. [2] (RAGAs)	2024	RAG evaluation	Reference-free RAG metrics	Faithfulness and relevance metrics
Saad-Falcon et al. [3] (ARES)	2024	RAG evaluation	Automated judges with limited human labels	Calibration with a human subset
Ru et al. [4] (RAGChecker)	2024	RAG diagnosis	Separate retrieval and generation diagnostics	Component-level failure analysis
Liu et al. [5] (AgentBench)	2024	Agent evaluation	Multi-environment agent benchmark	Tool-use and task metrics
Zheng et al. [6]	2023	LLM-as-a-Judge	Judging with LLMs; bias discussion	Model-based evaluation with caution
Asai et al. [7] (Self-RAG)	2024	Self-reflective RAG	Retrieve, generate and critique	Retrieval need and support as judgements
Ma et al. [8] (MAESTRO)	2026	Testing and observability	Reliability and observability of agent systems	Connects tracing with evaluation

Table 2.1 Comparison of the reviewed studies

2.11 LITERATURE SYNTHESIS AND RESEARCH GAP

The reviewed work shows that RAG provides external knowledge [1], that RAG quality can be measured along several independent dimensions [2], [3], [4], that agents must be evaluated over multi-step behaviour [5], that LLM judges are useful but biased [6], and that reliability and observability are active research topics [7], [8]. These contributions are largely separate: evaluation frameworks score outputs, benchmarks test agents in fixed environments, and observability tools record execution.

The gap addressed by this project is practical integration. A developer who wants to improve a RAG-based agent needs, in one place, a repeatable dataset, a trace of every step, retrieval and answer metrics, tool-use metrics, prompt and model comparison, and failure diagnosis. The proposed platform contributes an integrated, lightweight implementation and evaluation workflow around a practical RAG application. It does not claim that the underlying ideas are individually new.

CHAPTER 3
SYSTEM ANALYSIS

3.1 OVERVIEW OF THE EXISTING SYSTEM

In a conventional LLM or RAG application, a user submits a query and receives an answer, and the developer judges informally whether it looks correct. When a prompt, model or retrieval setting is changed, a few queries are tried again by hand. Logging, if present, is limited to application logs and does not capture retrieved chunks, tool arguments or step-wise timing in a structured form.

3.2 EXISTING-SYSTEM WORKFLOW

The existing workflow is linear and opaque: User Query → LLM or RAG pipeline → Answer → Manual human check. The intermediate stages are not recorded, so the checker sees only the final text.

3.3 DISADVANTAGES OF THE EXISTING SYSTEM

- Evaluation is manual, subjective and cannot be repeated consistently.
- Retrieval errors and generation errors are not separated.
- Tool calls made by an agent are not visible, so wrong selections go unnoticed.
- Hallucinated or unsupported statements may not be detected.
- It is difficult to identify the execution step at which an error occurred.
- Prompt versions and models cannot be compared on the same test cases.
- Latency, token usage and cost are not tracked per step.
- There is no regression testing after a change and no centralised dashboard.

3.4 PROPOSED SYSTEM

The proposed system places an evaluation and observability layer around an AI-agent application. The platform executes a RAG-based agent over a repeatable evaluation dataset, records an execution trace for every run, evaluates the retrieved context and the generated answer, monitors tool use and performance, and presents the results through a dashboard. Failed or low-scoring runs receive a failure category, and the same dataset is re-run as a regression test.

The platform does not assume that an automated score is always correct. It combines deterministic metrics, model-based evaluation and a small human-validated dataset used for calibration. The features described in this chapter are planned design features and are not presented as completed implementation.

3.5 ADVANTAGES OF THE PROPOSED SYSTEM

- Repeatable evaluation datasets allow fair comparison of prompts and models.
- Execution traces expose retrieval results, tool calls, timing and errors for every run.
- Retrieval evaluation and answer evaluation are reported separately.

- Tool-use monitoring measures tool-selection correctness and failure rate.
- Failure diagnosis classifies problems into retrieval, tool, generation, prompt or infrastructure causes.
- Regression testing detects performance drops; the open-source stack can run locally with small models.

3.6 FUNCTIONAL REQUIREMENTS

Table 3.1 Functional requirements

ID	Requirement
FR1	The system shall accept a user query and return an answer with source references.
FR2	The system shall ingest documents, split them into chunks and build a vector index.
FR3	The system shall execute an agent that can retrieve context and call optional tools.
FR4	The system shall record a trace containing the query, retrieved chunks, tool calls, response, latency, tokens and errors for each run.
FR5	The system shall store evaluation datasets with expected answers, expected documents and expected tools.
FR6	The system shall compute retrieval, answer and agent metrics for each trace.
FR7	The system shall allow experiments with different models, prompt versions and retrieval settings.
FR8	The system shall display traces, metrics, comparisons and failure categories on a dashboard.
FR9	The system shall allow a stored dataset to be re-run for regression testing.

3.7 NON-FUNCTIONAL REQUIREMENTS

- Reproducibility: the same dataset, configuration and versions shall be recorded with every experiment.
- Modularity: retrieval, agent, evaluation and dashboard components shall be replaceable.
- Performance and usability: evaluation shall complete in reasonable time on a standard machine and results shall be readable without raw logs.
- Privacy: documents and queries shall remain local when local models are used.
- Reliability and maintainability: a failed tool or model call shall be logged without stopping the run, and configuration shall be tracked in version control.

3.8 USER ROLES AND SYSTEM USE

Three roles are considered: the End User, who submits queries and reads answers with sources; the Developer or Evaluator, who manages datasets, runs evaluations, inspects traces, compares prompts and models and views the dashboard; and the Administrator, an optional role who manages knowledge documents and configures models and prompts. In a small team one person may hold all three roles.

CHAPTER 4
SYSTEM SPECIFICATION

4.1 HARDWARE REQUIREMENTS

The platform is designed to be developed on a standard laptop or desktop. A GPU is optional and is needed only when larger local models are used. The actual specification of the development machine is to be recorded below.

Component	Requirement
Processor	Intel Core i5 or equivalent (or above)
RAM	8 GB minimum; 16 GB recommended for local model inference
Storage	SSD with 20 GB or more free space
GPU	Optional NVIDIA GPU for faster local inference
Actual development machine	[INSERT ACTUAL DEVELOPMENT MACHINE SPECIFICATIONS]

Table 4.1 Hardware requirements

4.2 SOFTWARE REQUIREMENTS

The core selected stack is listed in Table 4.2. Alternatives are noted separately in Section 4.7; not every alternative is used at the same time.

Category	Technology	Purpose
Frontend	React with TypeScript	Web interface and dashboard
Backend	FastAPI with Pydantic	REST API and request validation
Agent orchestration	LangGraph	Agent workflow and state management
Embeddings	Sentence Transformers	Text embedding generation
Vector storage	FAISS (or Chroma)	Similarity search over chunk vectors
Model inference	Ollama and/or Hugging Face models	Local or API-based LLM inference
Database	SQLite	Traces, evaluations, experiments, datasets
Observability	MLflow (or Langfuse)	Trace and experiment tracking
Visualisation	Recharts	Charts in the dashboard
Version control	Git and GitHub	Source and configuration management

Table 4.2 Core software stack

4.3 FRONTEND REQUIREMENTS

The frontend is a single-page application written in TypeScript with React. It provides a chat interface for submitting queries, an experiment manager for selecting datasets, models and prompt versions, a trace viewer, and dashboard pages with charts. The interface communicates with the backend through JSON over HTTP. Recharts is used to render metric comparisons and latency charts.

4.4 BACKEND REQUIREMENTS

The backend is written in Python using FastAPI. Pydantic models define and validate the request and response schemas, including the trace record and evaluation result structures. The backend exposes endpoints for query submission, dataset management, experiment execution, trace retrieval and metric retrieval, and it invokes the LangGraph agent controller for every run.

4.5 AI AND RAG COMPONENTS

- Document loader and text cleaner for the knowledge base.
- Chunking module that attaches metadata such as document ID, name, section and page.
- Sentence Transformers model for embeddings, with FAISS or Chroma for indexing and top-k search.
- Optional reranking or hybrid keyword retrieval, to be added if time permits.
- LangGraph agent controller that decides on retrieval and tool calls.
- LLM inference through Ollama and/or Hugging Face models; a separate evaluator model is used for LLM-as-a-Judge where appropriate.

4.6 DATABASE AND LOGGING REQUIREMENTS

SQLite is the core relational database. It stores users, experiments, test cases, traces, tool calls, retrieved chunks, evaluations and document metadata (Section 6.11). The vector store is kept separately in FAISS or Chroma. Every run is logged with a Run ID, the model name, the prompt version and timestamps, so that any result can be reproduced and audited. MLflow or Langfuse may be used in addition to SQLite for trace visualisation and experiment tracking.

4.7 DEVELOPMENT AND OPTIONAL DEPLOYMENT TOOLS

Alternatives and optional items are summarised in Table 4.3. They are not all used simultaneously.

Item	Option	Status
Vector store	Chroma instead of FAISS	Alternative
Observability	Langfuse instead of MLflow	Alternative
Tools	Web search, calendar, college database, custom functions	Optional
Deployment	Docker containers	Optional, if required
Model hosting	Hugging Face hosted or API models	Alternative to local Ollama

Table 4.3 Alternatives and optional tools

CHAPTER 5
IMPLEMENTATION AND METHODOLOGY

5.1 OVERALL METHODOLOGY

The methodology follows a controlled lifecycle from knowledge-base preparation to repeatable evaluation: text processing, chunking, embeddings, vector indexing, query processing, retrieval, agent orchestration, optional tool use, LLM response, trace storage, evaluation, dashboard visualisation and failure analysis (Figure 5.1).

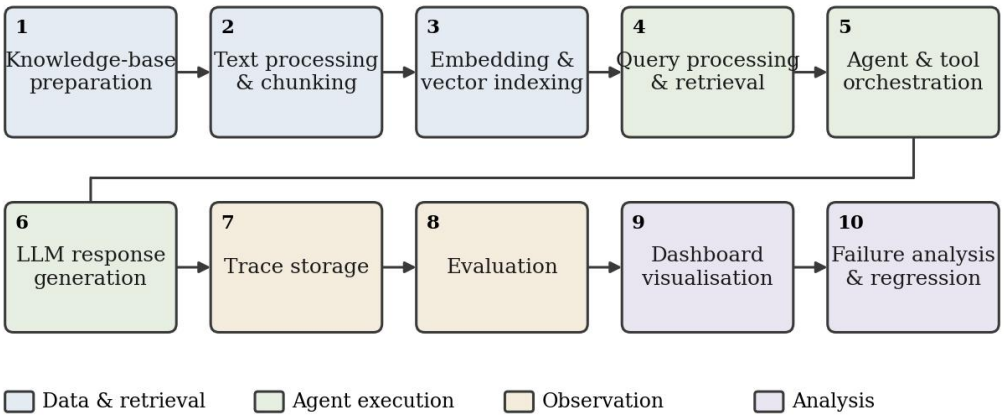


Figure 5.1 Methodology lifecycle of the proposed platform

Each stage passes its output to the next: indexed vectors serve retrieval, retrieval results and tool outputs feed the LLM, the stored trace feeds evaluation, and the evaluation results feed the dashboard and failure analysis. After a change, the same dataset is re-run, which closes the loop.

5.2 DATASET AND DOCUMENT PREPARATION

A small knowledge base is created for the selected application domain, for example public academic documents, college and department information, rules and regulations, and event information. For every document, metadata is recorded: document ID, name, section, page, category and source, so that each retrieved chunk can be traced to its origin and retrieval can be measured against expected documents.

An initial evaluation dataset of about 100 test cases is the target of this project; the final size will be stated after the dataset is completed. The dataset is planned to include the categories listed in Table 5.1. Each test case contains a test ID, question, expected answer, expected documents, expected tool, difficulty and category.

Category	Purpose
Normal	Straightforward questions answerable from one document
Multi-step	Questions needing several retrievals or tool calls
Retrieval-sensitive	Questions whose answer depends on retrieving the correct chunk
Ambiguous	Questions with unclear intent to test clarification or cautious answers
Irrelevant	Questions outside the knowledge base to test refusal or abstention

Category	Purpose
Tool-use	Questions requiring a specific tool such as a timetable or calendar lookup
Challenging / adversarial	Questions designed to induce unsupported or misleading answers

Table 5.1 Planned categories of evaluation queries

5.3 TEXT EXTRACTION, CLEANING AND CHUNKING

Text is extracted from the source documents, and headers, footers and repeated whitespace are removed. The cleaned text is divided into chunks, optionally with a small overlap so that sentences are not cut at boundaries. Chunk size and overlap are recorded with each experiment because they influence retrieval quality, and each chunk inherits its document metadata and receives a unique chunk ID.

5.4 EMBEDDING GENERATION AND INDEXING

Each chunk is converted into a dense vector using a Sentence Transformers model. The vectors are stored in a FAISS or Chroma index together with their chunk identifiers. The embedding model name is stored in the experiment configuration so that an index can be rebuilt identically. Indexing runs offline and is repeated only when the documents or the embedding model change (part A of Figure 5.2).

5.5 RETRIEVAL PROCESS

At query time the question is embedded with the same model and the top-k most similar chunks are retrieved by similarity search. Optional reranking or hybrid keyword retrieval may refine the ranking. The retrieved chunks and their scores are assembled into the context supplied to the language model. The complete pipeline is shown in Figure 5.2.

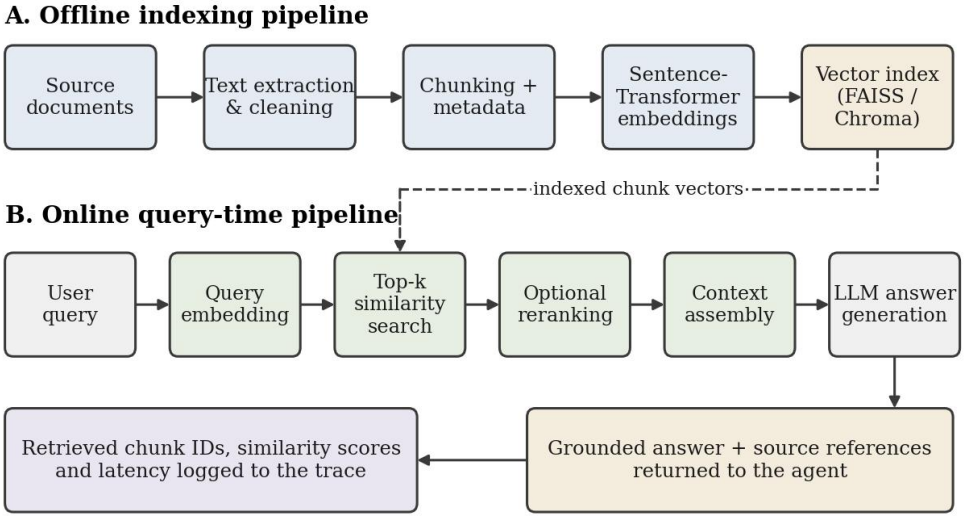


Figure 5.2 RAG pipeline: offline indexing and online query-time retrieval

Part A prepares the vector index from the source documents. Part B embeds the user query, searches the index, optionally reranks the results and assembles the context for the

LLM. The retrieved chunk IDs, scores and retrieval latency are written to the trace, so retrieval quality can be evaluated independently of the generated answer.

5.6 AI AGENT EXECUTION

The agent controller is implemented as a LangGraph workflow. On receiving a query the agent analyses the intent, retrieves context, decides whether a tool is required, executes the tool if needed, and then asks the LLM to generate the answer using the context and any tool result. For a simple question the flow is: query, agent analysis, retrieval, answer generation and answer with sources. For a tool-based query the flow is: question, agent, tool selection, tool execution, result, agent and final answer. Every execution receives a unique Run ID.

5.7 TOOL EXECUTION

Tools are registered as functions with a name, a description and a typed argument schema. Examples are web search, a calendar lookup, a college database query and custom calculation functions such as attendance calculation. When the agent selects a tool, the tool name, input arguments, output, status and latency are recorded. A failed tool call is caught, logged and returned to the agent so that it can retry or continue. The tool layer is optional.

5.8 TRACE COLLECTION

The trace collector records, for every run, the Run ID and timestamp, the query and prompt version, the retrieved chunk IDs and similarity scores, the model name, token usage, generation latency, tool names, parameters and results, errors, the final response and, later, the evaluation result. The trace is stored in SQLite and may also be logged to MLflow or Langfuse. Figure 5.3 shows the execution flow and the trace record captured at each step.

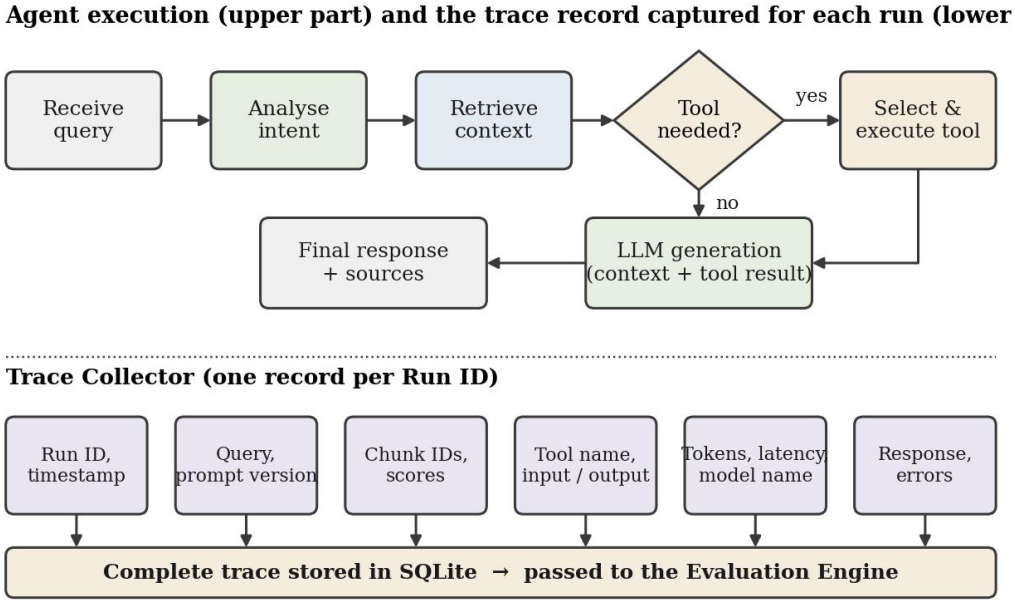


Figure 5.3 Agent execution flow and trace-collection record

The upper part shows the agent path from the received query to the final response, including the optional tool branch. The lower part shows the items captured by the trace collector for each run. Because the trace stores how an answer was produced and not only the answer, a developer can locate the step that caused a failure.

5.9 EVALUATION ENGINE

The Evaluation Engine reads each stored trace and computes metrics using a hybrid strategy, so that no single automated metric is treated as absolute ground truth. Deterministic evaluation covers exact or keyword match where suitable, Recall@K, tool-selection accuracy, task completion, latency and failure rate. Automated model-based evaluation uses an LLM-as-a-Judge to score answer quality, relevance, faithfulness and completeness, preferably with an evaluator model different from the generating model. Human validation uses a small manually reviewed subset to calibrate and sanity-check the automated scores. Definitions are given in Chapter 7 and Appendix A.

5.10 DASHBOARD AND EXPERIMENT COMPARISON

The dashboard presents metric cards, comparison charts for prompt and model versions, latency per step, a trace table and failure categories. An experiment records the model, prompt version and retrieval configuration, so the same dataset can be compared across configurations. Figure 5.4 is an illustrative design mock-up, not a screenshot of a working application.

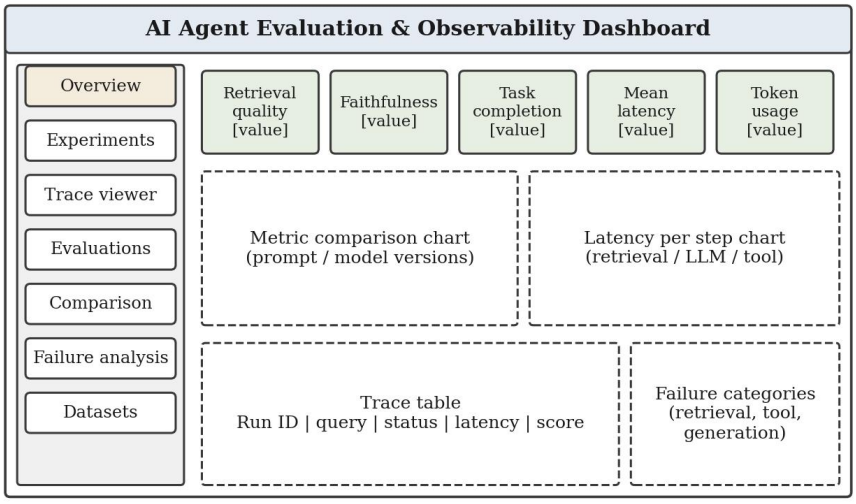


Figure 5.4 Illustrative dashboard layout (design mock-up, not an implementation screenshot)

The layout groups navigation on the left, summary metric cards at the top, comparison and latency charts in the middle, and the trace table and failure categories at the bottom. Values shown as [value] will be supplied by the evaluation engine after experiments are executed.

CHAPTER 6
SYSTEM DESIGN

6.1 INTRODUCTION TO SYSTEM DESIGN

This chapter presents the design of the proposed platform using architecture, data flow, UML and entity-relationship diagrams. The diagrams describe the planned system and are not a record of a completed implementation. Each figure is followed by a short explanation.

6.2 OVERALL SYSTEM ARCHITECTURE

The platform is organised into five layers (Figure 6.1): presentation (React and TypeScript), application (FastAPI), intelligence (LangGraph agent controller, RAG engine, embedding model, LLM inference and optional tools), evaluation and observability, and data (SQLite, vector store and documents).

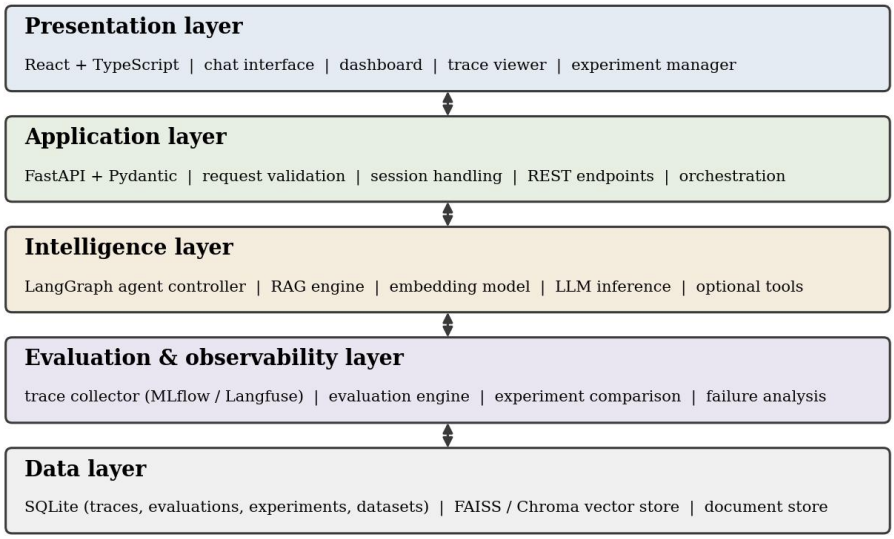


Figure 6.1 Layered architecture of the platform

Requests move downwards to the intelligence layer and results move back upwards. The evaluation and observability layer observes the intelligence layer and stores its results in the data layer, which keeps evaluation separate from the application logic being evaluated.

6.3 SYSTEM ARCHITECTURE / BLOCK DIAGRAM

Figure 6.2 shows the logical components. The user works through the React and TypeScript frontend, which calls the FastAPI backend and then the LangGraph agent controller. The controller coordinates the RAG engine, embedding model, vector store, LLM inference layer and optional tools. Execution events flow to the trace collector, the evaluation engine and SQLite, and the dashboard reads the results through the backend.

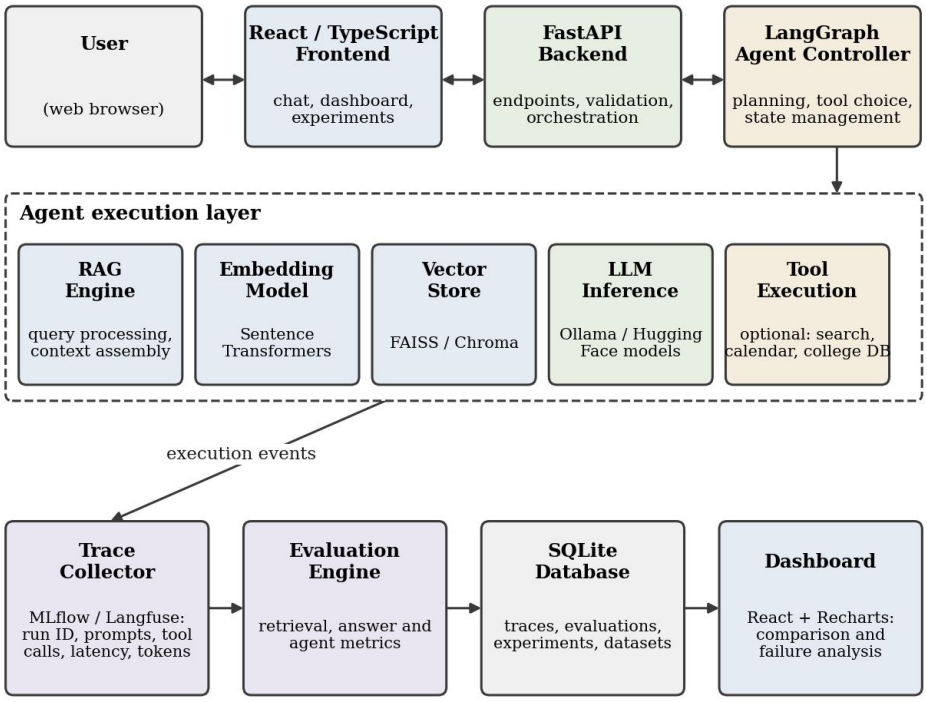


Figure 6.2 System architecture / block diagram

The dashed container groups the components coordinated by the agent controller. The lower row is the observation path. SQLite is the core database and FAISS or Chroma is the vector store, as selected in Chapter 4.

6.4 CONTEXT-LEVEL DATA FLOW DIAGRAM (DFD LEVEL 0)

Figure 6.3 treats the platform as one process. The user sends queries and receives answers with sources; the developer or evaluator supplies datasets and configurations and receives traces, metrics and reports. External entities are the knowledge documents, external tools or APIs, and the LLM inference service.

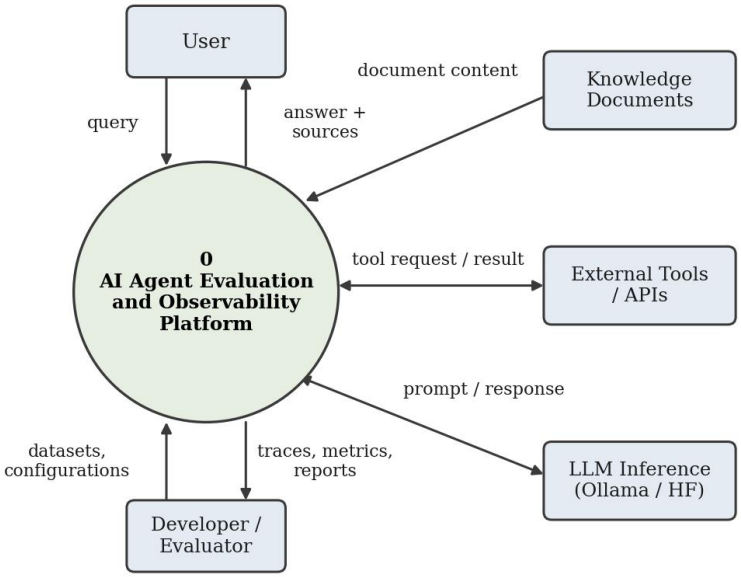


Figure 6.3 Context-level data flow diagram (DFD Level 0)

The diagram defines the system boundary and the data crossing it. Internal data stores appear in the Level 1 diagram.

6.5 DETAILED DATA FLOW DIAGRAM (DFD LEVEL 1)

Figure 6.4 has seven processes and three data stores. Process 1.0 handles the query and 2.0 retrieves context from the vector store (D1). Process 3.0 executes the agent and tools, 4.0 generates the response and 5.0 collects the trace into the SQLite database (D2). Process 6.0 evaluates the response using the evaluation dataset (D3), and 7.0 visualises and compares the results.

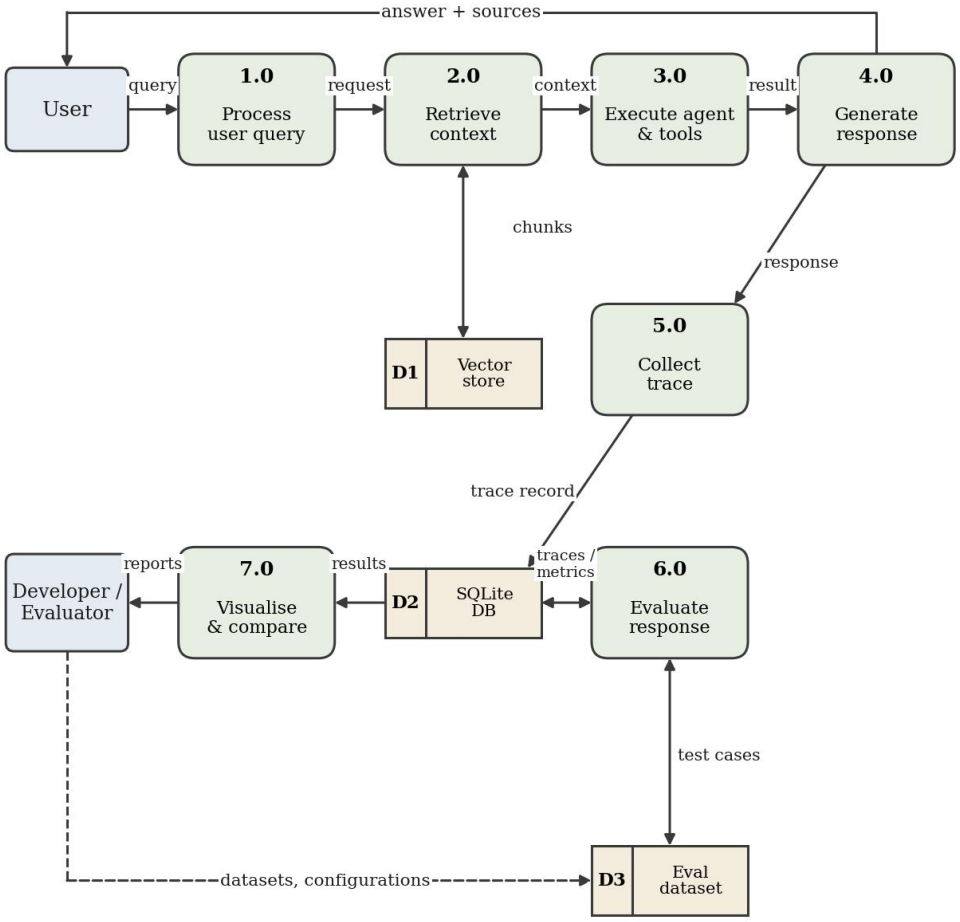


Figure 6.4 Detailed data flow diagram (DFD Level 1)

Rounded rectangles are processes, open rectangles are data stores and plain boxes are external entities. The trace record links the application flow (1.0 to 4.0) with the evaluation flow (5.0 to 7.0).

6.6 USE CASE DIAGRAM

Figure 6.5 shows three actors. The End User submits queries and views responses and sources. The Developer or Evaluator manages the dataset, runs evaluations, inspects traces, compares prompts and models and views the dashboard. The Administrator, an optional role, manages knowledge documents and configures models and prompts.

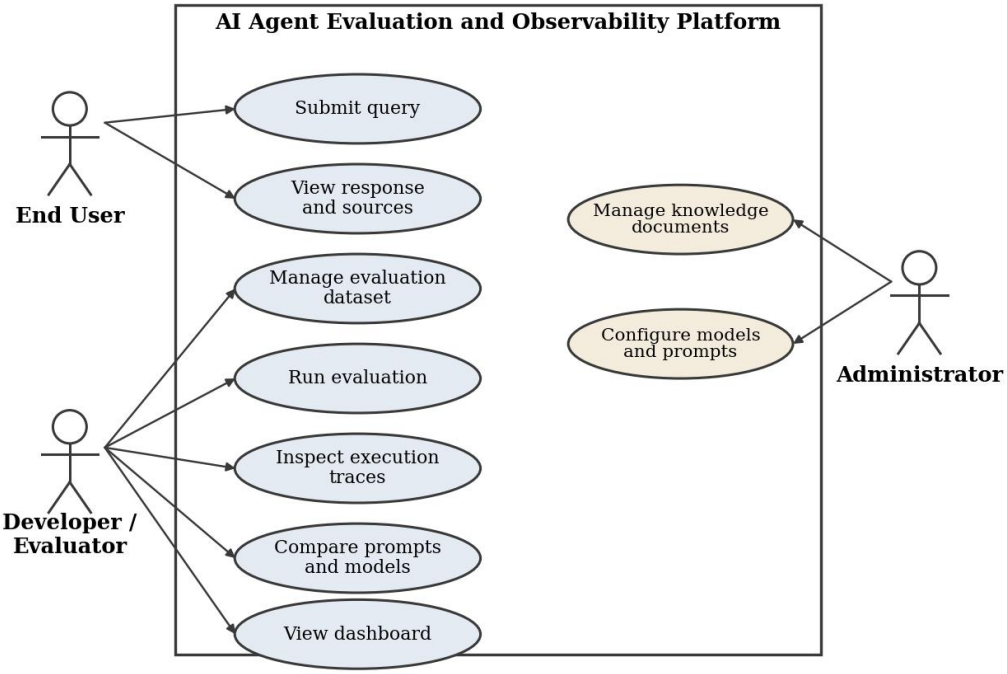


Figure 6.5 Use case diagram

The system boundary contains the use cases. Administrator functions are optional design extensions, and in a small deployment the Developer or Evaluator may also act as Administrator.

6.7 SEQUENCE DIAGRAM

Figure 6.6 shows one query. The frontend sends the request to the backend, which starts an agent run. The agent retrieves context from the RAG engine, optionally calls a tool, prompts the LLM with the context, stores the trace and returns the response. Evaluation is performed afterwards.

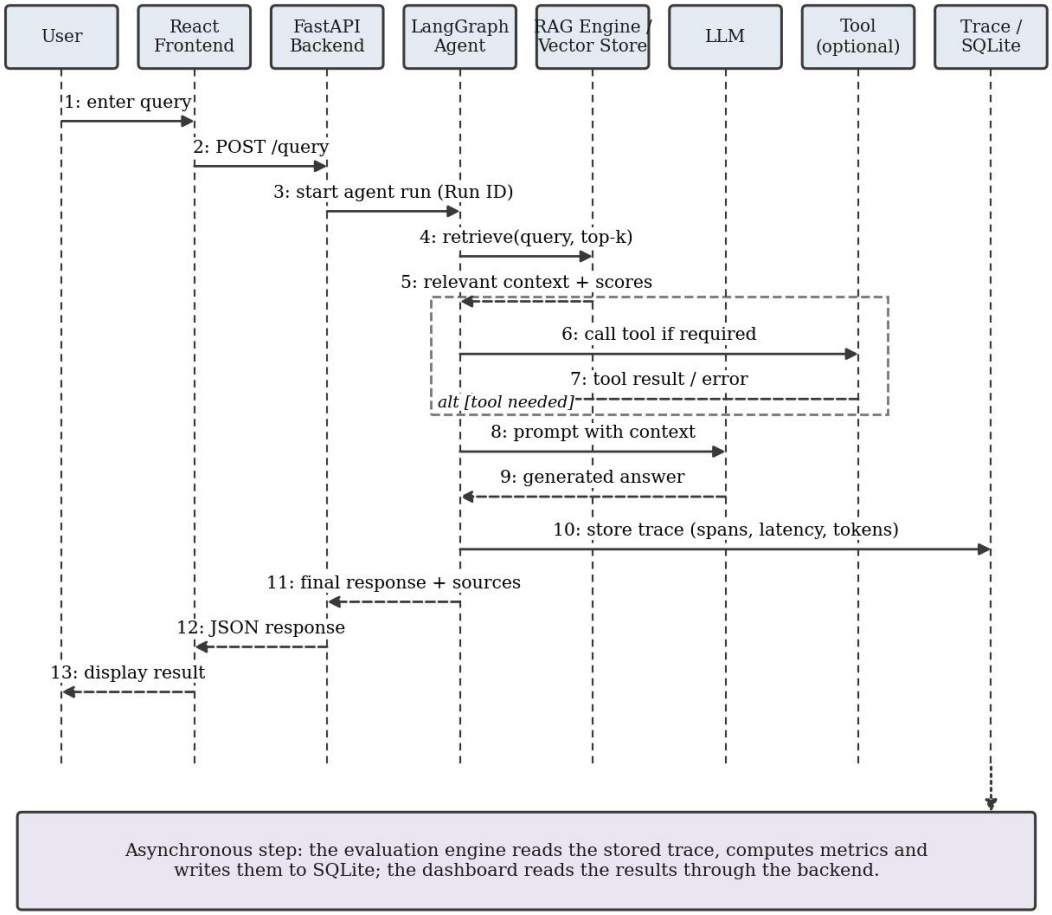


Figure 6.6 Sequence diagram of a RAG and agent execution

Solid arrows are calls and dashed arrows are returns. The alternative fragment marks the optional tool call. Trace storage happens within the run, while evaluation is asynchronous.

6.8 ACTIVITY DIAGRAM

Figure 6.7 shows the normal path and the failure paths: retrieval failure, tool failure and runs whose metrics do not meet the threshold. Every path stores a trace, so failures are available for analysis.

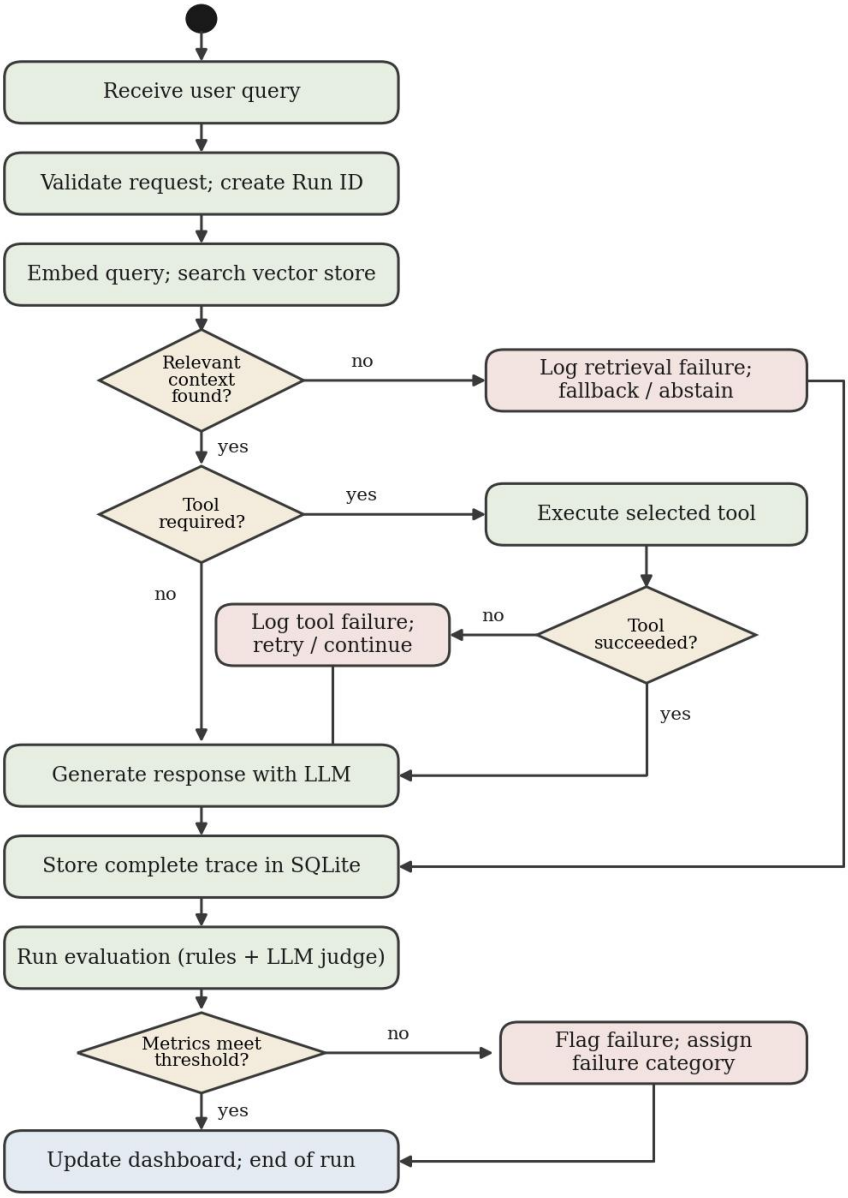


Figure 6.7 Activity diagram with normal and failure paths

Diamonds are decisions and light-red boxes mark failure handling. Flagged runs receive a failure category before the dashboard is updated.

6.9 ENTITY RELATIONSHIP DIAGRAM (ERD)

Figure 6.8 shows the proposed logical database design. A user runs many experiments, an experiment produces many traces, and each trace belongs to one test case. A trace invokes many tool calls, retrieves many chunks and is scored by many evaluations; each retrieved chunk refers to one document. The design will be updated to match the final implementation.

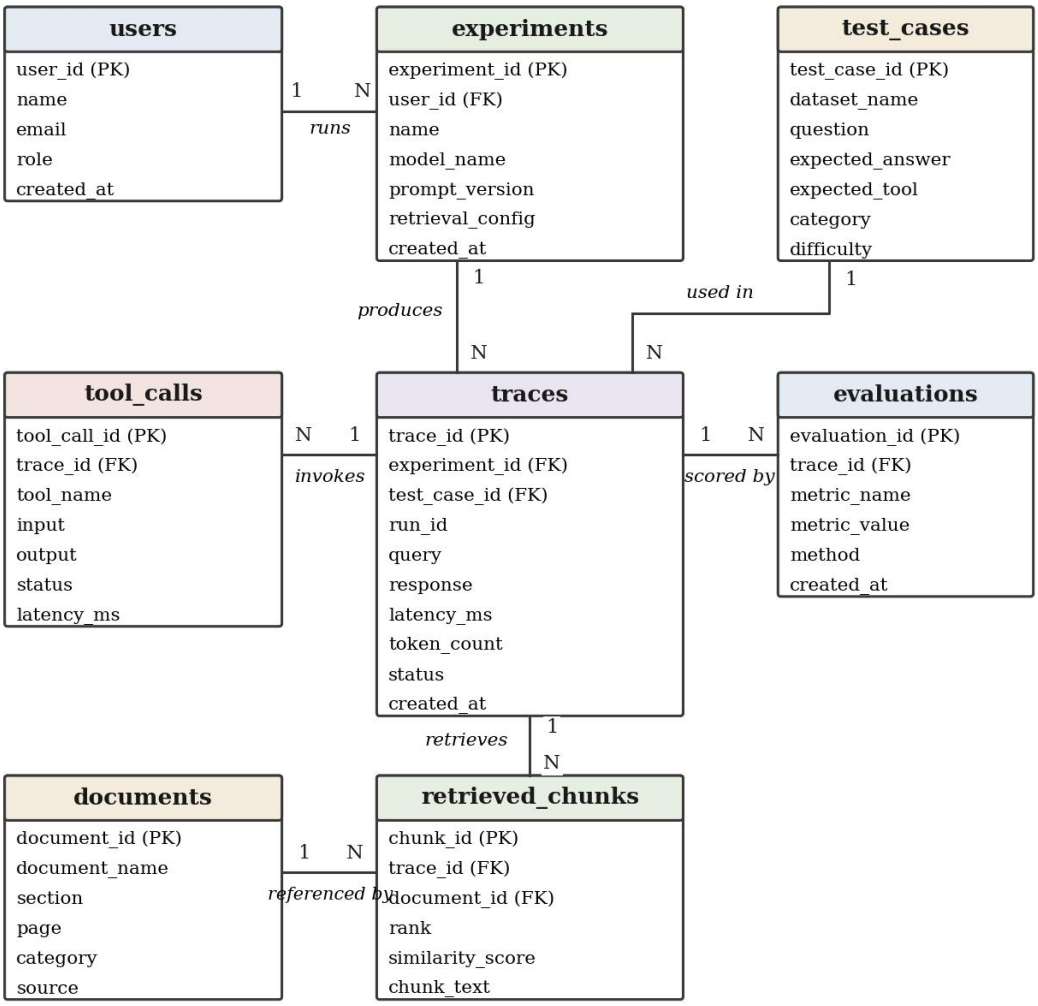


Figure 6.8 Entity relationship diagram (proposed logical design)

PK is a primary key and FK a foreign key; 1 to N means one row of the left entity relates to many rows of the right entity. The retrieved_chunks entity resolves the many-to-many relationship between traces and documents.

6.10 EVALUATION PIPELINE AND TRACE LIFECYCLE

A trace begins when a run is created and ends when its metrics are shown on the dashboard (Figure 6.9). It is evaluated by deterministic checks, model-based evaluation and a human-validated subset; the metrics are stored per trace and per experiment, failures are categorised, and improvements are tested by re-running the same dataset.

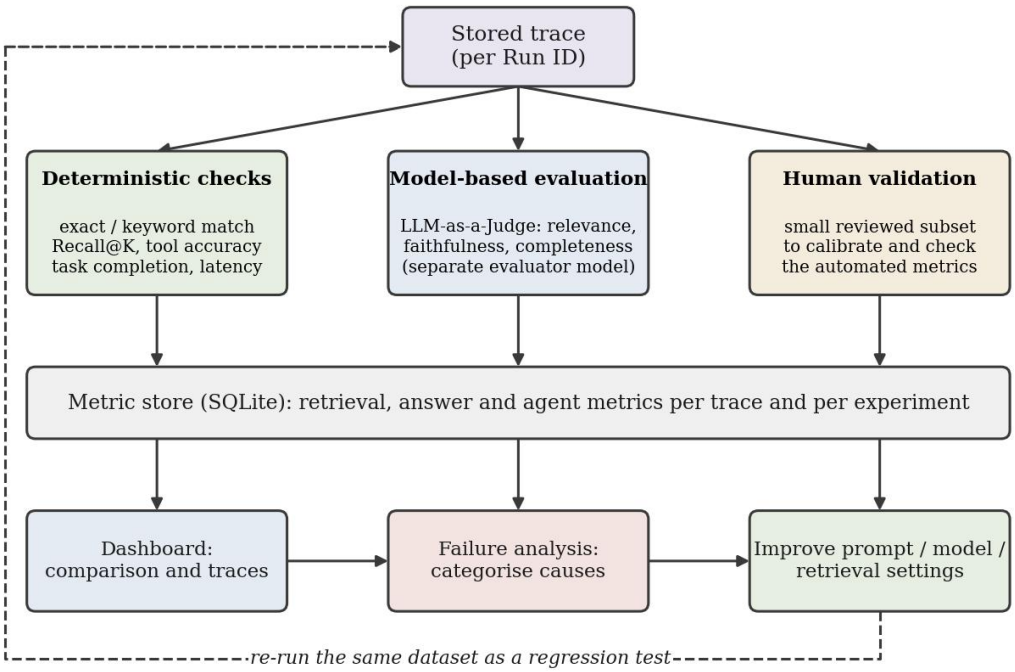


Figure 6.9 Evaluation and observability lifecycle

The three evaluation methods feed one metric store. The dashed arrow represents regression testing on the same dataset after a change.

6.11 DATABASE DESIGN

The proposed tables are summarised in Table 6.1. Tool calls and retrieved chunks are kept in separate tables linked to a trace, so a single trace can have any number of each.

Table 6.1 Proposed database tables

Table	Primary key	Foreign keys	Main attributes
users	user_id	-	name, email, role, created_at
experiments	experiment_id	user_id	name, model_name, prompt_version, retrieval_config, created_at
test_cases	test_case_id	-	dataset_name, question, expected_answer, expected_tool, category, difficulty
traces	trace_id	experiment_id, test_case_id	run_id, query, response, latency_ms, token_count, status, created_at
tool_calls	tool_call_id	trace_id	tool_name, input, output, status, latency_ms
retrieved_chunks	chunk_id	trace_id, document_id	rank, similarity_score, chunk_text
documents	document_id	-	document_name, section, page, category, source
evaluations	evaluation_id	trace_id	metric_name, metric_value, method, created_at

6.12 MODULE DESCRIPTION

Table 6.2 Modules of the platform

Module	Responsibility	Main output

Module	Responsibility	Main output
User Interface	Chat, experiment manager, dashboard and trace viewer	Queries; displayed results
Backend API	Request validation, routing and run orchestration	JSON responses; run records
Agent Controller	Planning, tool selection and state handling in LangGraph	Agent decisions; final answer
RAG Engine	Ingestion, chunking, embedding, indexing and retrieval	Ranked context chunks
Trace Collector	Recording of every step of a run	Trace records
Evaluation Engine	Deterministic, model-based and human-calibrated metrics	Metric values per trace
Experiment Manager	Configuration and comparison of prompts, models and settings	Experiment results
Analytics Dashboard	Visualisation of metrics, traces and failures	Charts and reports

CHAPTER 7
SYSTEM TESTING AND EVALUATION

7.1 TESTING OVERVIEW

Testing covers the platform's own software and the quality of the RAG-based agent that it evaluates. The tables in this chapter are a proposed test plan. No test has yet been executed, so the actual outcome and status columns are marked [FILL AFTER EXECUTION].

7.2 UNIT TESTING

Unit tests check individual functions in isolation: chunking, embedding and similarity search, metric functions and trace record validators.

Table 7.1 Proposed unit test cases

Test ID	Scenario / input	Expected outcome	Actual outcome	Status
UT-01	Chunk a document with a given size and overlap	Chunks respect size limit and carry metadata	[FILL AFTER EXECUTION]	[FILL AFTER EXECUTION]
UT-02	Embed a query and search a small index	Top-k chunk IDs returned in score order	[FILL AFTER EXECUTION]	[FILL AFTER EXECUTION]
UT-03	Compute Recall@K for a known example	Value equals hand-calculated result	[FILL AFTER EXECUTION]	[FILL AFTER EXECUTION]
UT-04	Validate a trace record with a missing field	Pydantic validation error is raised	[FILL AFTER EXECUTION]	[FILL AFTER EXECUTION]

7.3 INTEGRATION TESTING

Integration tests check that modules work together: backend with agent, agent with RAG engine and tools, trace collector with database, and evaluation engine with stored traces.

Table 7.2 Proposed integration test cases

Test ID	Scenario / input	Expected outcome	Actual outcome	Status
IT-01	Submit a query through the API	Answer, sources and Run ID returned	[FILL AFTER EXECUTION]	[FILL AFTER EXECUTION]
IT-02	Run an agent query that needs a tool	Tool call recorded with input, output and status	[FILL AFTER EXECUTION]	[FILL AFTER EXECUTION]
IT-03	Run a query with no relevant document	Retrieval failure logged; run completes	[FILL AFTER EXECUTION]	[FILL AFTER EXECUTION]
IT-04	Evaluate a stored trace	Metrics written to the evaluations table	[FILL AFTER EXECUTION]	[FILL AFTER EXECUTION]

7.4 FUNCTIONAL TESTING

Functional tests check the requirements of Section 3.6 from the user's point of view.

Table 7.3 Proposed functional test cases

Test ID	Scenario / input	Expected outcome	Actual outcome	Status
FT-01	Ask a normal question in the chat interface	Relevant answer with source references	[FILL AFTER EXECUTION]	[FILL AFTER EXECUTION]
FT-02	Run an experiment with two prompt versions	Two result sets stored and comparable	[FILL AFTER EXECUTION]	[FILL AFTER EXECUTION]
FT-03	Open the trace viewer for a run	Retrieved chunks, tool calls and timings shown	[FILL AFTER EXECUTION]	[FILL AFTER EXECUTION]

7.5 SYSTEM TESTING

System testing runs the complete workflow, from document ingestion to dashboard display, on the full evaluation dataset. It checks that the pipeline completes, that one failing test case does not stop the run, and that stored results are consistent.

7.6 EVALUATION METRICS

The metrics in Table 7.4 are used to evaluate the RAG-based agent. Deterministic metrics are computed by rules from labelled data and are repeatable. Model-based metrics are produced by an evaluator model and depend on the model and its prompt, and therefore need validation. Formal definitions are given in Appendix A.

Table 7.4 Evaluation metrics, purpose and limitations

Metric	Type	Purpose	Limitation
Retrieval quality / Recall@K	Deterministic	Whether expected documents appear in the top K	Needs ground-truth labels
Answer correctness	Deterministic / model-based	Agreement with the expected answer	Exact match is strict; judges may be lenient
Answer relevance	Model-based	Whether the answer addresses the question	Depends on the judge model
Faithfulness	Model-based	Whether claims are supported by the retrieved context	Claim extraction can be imperfect
Tool-selection correctness	Deterministic	Whether the expected tool was chosen	Several tools may be acceptable
Task completion	Deterministic / model-based	Whether the task was completed	Needs a clear success definition
Latency	Deterministic	Total and per-step response time	Depends on hardware and load
Token usage	Deterministic	Prompt and completion tokens per run	Tokeniser differs by model
Error / failure rate	Deterministic	Share of runs with errors or failed tools	Needs a failure taxonomy
Prompt / model comparison	Aggregate	Compare configurations on one dataset	Valid only for that dataset

7.7 TEST-CASE DESIGN

Evaluation test cases are designed from the categories in Table 5.1; each specifies the question, expected answer, expected documents and expected tool. The cases below illustrate the structure.

Test ID	Category / scenario	Expected outcome	Actual outcome	Status
EV-01	Normal: single-document question	Correct answer; expected document retrieved	[FILL AFTER EXECUTION]	[FILL AFTER EXECUTION]
EV-02	Multi-step question	All steps completed; correct final answer	[FILL AFTER EXECUTION]	[FILL AFTER EXECUTION]
EV-03	Irrelevant question	Answer states that information is unavailable	[FILL AFTER EXECUTION]	[FILL AFTER EXECUTION]
EV-05	Adversarial question	No unsupported claim in the answer	[FILL AFTER EXECUTION]	[FILL AFTER EXECUTION]

Table 7.5 Proposed evaluation test cases (structure)

7.8 FAILURE ANALYSIS

A failed test is classified from its trace into a category in Table 7.6, which determines the next action; the dataset is then re-run as a regression test.

Table 7.6 Failure categories

Category	Typical evidence in the trace	Possible action
Retrieval failure	Expected document absent from retrieved chunks	Adjust chunking, embedding or top-k
Tool-selection failure	Wrong or no tool called	Improve tool descriptions or prompt
Tool-execution failure	Tool call status is error	Fix tool or add retry logic
Unsupported generation	Claims not found in retrieved context	Strengthen grounding instructions
Prompt regression	Metric drop after a prompt change	Revert or revise the prompt
Infrastructure failure	Timeouts or model errors	Check resources and configuration

7.9 RESULT PRESENTATION

Results will be presented as dashboard tables and charts: per-experiment metrics, prompt and model comparisons, latency per step and failure counts. No numerical results are stated in this report; values will be reported only after execution, with the dataset size and configuration.

CHAPTER 8
FEASIBILITY STUDY, WORK PLAN AND LIMITATIONS

8.1 TECHNICAL FEASIBILITY

All components of the selected stack are open-source and widely documented. Local inference with small models through Ollama or Hugging Face is possible on a standard development machine, and SQLite and FAISS need no separate server. The team's previous work with Python, FastAPI and React supports the technical feasibility.

8.2 ECONOMIC FEASIBILITY

The platform relies on free and open-source software and can run on existing hardware, so the development cost is mainly the time of the team. Optional hosted model APIs may add a usage cost, which is controlled by using local models for most experiments.

8.3 OPERATIONAL FEASIBILITY

The platform is operated through a web interface and a dashboard, so a developer or evaluator does not need to read raw logs. Configuration files and stored experiments allow the evaluation to be repeated by another team member.

8.4 DATA AND RESOURCE REQUIREMENTS

The project needs a small knowledge-base document collection with permission to use it, a labelled evaluation dataset of about 100 test cases, a development machine as described in Chapter 4, and the time of the team for dataset labelling and human validation.

8.5 TWELVE-WEEK WORK PLAN

The planned schedule is given in Table 8.1 and Figure 8.1. It is a planned work schedule and not a record of completed work.

Week	Planned work
Week 1	Problem identification and finalisation
Week 2	Literature survey
Week 3	Requirements and architecture
Week 4	Dataset and document preparation
Week 5	RAG implementation
Week 6	Agent implementation
Week 7	Trace collection
Week 8	Evaluation engine
Week 9	Dashboard
Week 10	Integration
Week 11	Testing and experiments
Week 12	Analysis and documentation

Table 8.1 Planned twelve-week work schedule

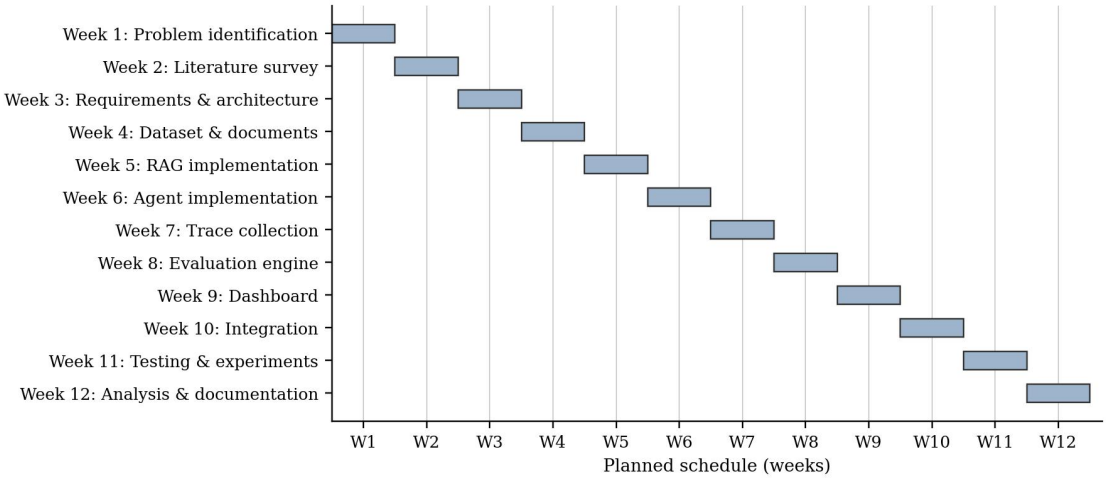


Figure 8.1 Planned twelve-week project work plan

Each bar shows the week planned for an activity. Dates and completion status are not shown because the schedule has not yet been executed.

8.6 SCOPE AND LIMITATIONS

Current scope: RAG-based AI agents, small and medium evaluation datasets, local or API-based LLMs, tool-use evaluation, retrieval evaluation, response evaluation, trace collection, dashboard visualisation, and prompt and model comparison. The following limitations apply:

- Automated and LLM-based evaluation may contain evaluator bias.
- Small datasets may not cover every real-world scenario.
- Local LLM quality and speed depend on the available hardware, and results from local models may differ from API-based models.
- Several metrics depend on labelled data, such as expected documents and expected tools.
- Observability helps to diagnose reliability problems but does not guarantee that the model's answers become correct.

8.7 RISK CONSIDERATIONS

Risk	Impact	Mitigation
Dataset labelling takes longer than planned	Delay in evaluation	Start with a smaller core dataset and extend it
LLM judge gives inconsistent scores	Unreliable metrics	Calibrate with a human-reviewed subset; fix judge prompt and model
Local hardware is too slow for the chosen model	Long evaluation runs	Use a smaller model or an API model for selected runs
Integration problems between components	Schedule slip	Use Pydantic schemas and integrate weekly
Tool or API unavailable	Failed runs	Treat tools as optional; log failures and continue

Table 8.2 Risks and mitigation

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENT

9.1 CONCLUSION

This project proposes an integrated platform for evaluating and observing RAG-based AI agents. Instead of treating an AI application as a black box, the platform is designed to capture retrieval operations, tool interactions, model responses and execution characteristics in a trace for every run. The evaluation engine measures retrieval quality, response quality and agent behaviour using deterministic checks, model-based evaluation and a small human-validated set, and the dashboard presents traces, comparisons and failure categories. Together these elements support systematic testing, prompt and model comparison, failure analysis and regression testing.

The report presents the design, methodology and test plan. Implementation results and measured metrics are to be added after the system is built and tested. The project also provides practical experience in RAG, AI agents, evaluation design, backend development, frontend visualisation and observability.

9.2 FUTURE ENHANCEMENT

- Multi-agent evaluation.
- Continuous evaluation integrated into CI/CD pipelines.
- Automated red-team testing of agents.
- Security evaluation of AI applications.
- Automated prompt optimisation.
- Model routing based on evaluation scores.
- Distributed tracing and production alerting.
- Larger-scale benchmark execution.

These enhancements are future work and are not part of the present design.

APPENDIX

A. ADDITIONAL EVALUATION METRIC DEFINITIONS

Recall@K = (relevant documents retrieved in the top K) / (total relevant documents)

Tool Selection Accuracy = (correct tool selections) / (total tool-selection cases)

Task Completion Rate = (successfully completed tasks) / (total tasks)

Tool Failure Rate = (failed tool calls / total tool calls) × 100

Faithfulness = (answer claims supported by the retrieved context) / (total answer claims)

B. DATASET FIELD DEFINITIONS

Table 0.1 Fields of an evaluation test case

Field	Description
test_id	Unique identifier of the test case
question	Query sent to the agent
expected_answer	Reference answer, where available
expected_documents	Documents that should be retrieved
expected_tool	Tool that should be selected, if any
difficulty	Easy, medium or hard
category	Normal, multi-step, ambiguous, irrelevant, tool-use, adversarial

C. SAMPLE TRACE STRUCTURE

Illustrative structure only; names are placeholders and values are not real results.

```
{
  "run_id": "<RUN-ID>",
  "query": "<user question>",
  "model": "<model name>", "prompt_version": "<version>",
  "retrieved": [...], "tool_calls": [...],
  "latency_ms": <value>, "tokens": <value>,
  "response": "<answer text>", "errors": []
}
```

D. IMPLEMENTATION SCREENSHOTS (TO BE INSERTED)

Actual screenshots will replace the placeholders below after the application is built.

[INSERT ACTUAL SYSTEM ARCHITECTURE SCREENSHOT HERE]

[INSERT ACTUAL DASHBOARD SCREENSHOT HERE]

[INSERT ACTUAL TRACE VIEW SCREENSHOT HERE]

REFERENCES

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020.

[2] S. Es, J. James, L. Espinosa Anke, and S. Schockaert, "RAGAs: Automated Evaluation of Retrieval Augmented Generation," in Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations, pp. 150–158, 2024, doi: 10.18653/v1/2024.eacl-demo.16.

[3] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, "ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems," in Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, pp. 338–354, 2024, doi: 10.18653/v1/2024.naacl-long.20.

[4] D. Ru, L. Qiu, X. Hu, T. Zhang, P. Shi, S. Chang, C. Jiayang, C. Wang, S. Sun, H. Li, Z. Zhang, B. Wang, J. Jiang, T. He, Z. Wang, P. Liu, Y. Zhang, and Z. Zhang, "RAGChecker: A Fine-grained Framework for Diagnosing Retrieval-Augmented Generation," Advances in Neural Information Processing Systems (NeurIPS), vol. 37, 2024, doi: 10.52202/079017-0692.

[5] X. Liu et al., "AgentBench: Evaluating LLMs as Agents," International Conference on Learning Representations (ICLR), 2024.

[6] L. Zheng et al., "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," Advances in Neural Information Processing Systems (NeurIPS), vol. 36, 2023, doi: 10.52202/075280-2020.

[7] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," International Conference on Learning Representations (ICLR), 2024.

[8] T. Ma, Y. Chen, V. Anand, A. Cornacchia, A. R. Faustino, G. Liu, S. Zhang, H. Luo, S. A. Fahmy, Z. A. Qazi, and M. Canini, "MAESTRO: Multi-Agent Evaluation Suite for Testing, Reliability, and Observability," arXiv:2601.00481, 2026.

BIBLIOGRAPHY

The official documentation of the selected tools was used as background: FastAPI and Pydantic, LangGraph, Sentence Transformers, FAISS and Chroma, Ollama and Hugging Face, MLflow and Langfuse, and React, TypeScript and Recharts